

	AU: 2026-2027 Ateliers SOA Et Service Web Enseignante :Bènène Fradi Boumiza
Atelier 1 : Génération de documents XML/XSD et java via JAXB (Marshalling et Unmarshalling)	

Introduction

Dans le développement des services web, les applications doivent souvent échanger des données entre des environnements et des technologies différents. Ces données sont généralement représentées sous forme de documents XML ou JSON. Pour pouvoir exploiter ces données dans une application Java, il est nécessaire d'assurer une correspondance entre les documents échangés et les objets Java utilisés par l'application.

Dans le cas des services web basés sur XML, notamment les services SOAP, cette correspondance est assurée par des mécanismes de binding, de marshalling et d'unmarshalling. Des frameworks et des outils de développement prennent généralement en charge ces opérations automatiquement, sans que le développeur ait à écrire manuellement tout le code de conversion.

Objectifs

- Mapper des classes Java vers des documents XML et inversement.
- Générer un document XML à partir d'objets Java (marshalling).
- Générer des objets Java à partir de documents XML (unmarshalling).
- Générer un schéma XSD à partir d'une classe Java.
- Générer des classes Java à partir d'un schéma XSD.
- Comprendre le rôle de JAXB dans les architectures orientées services (SOA).

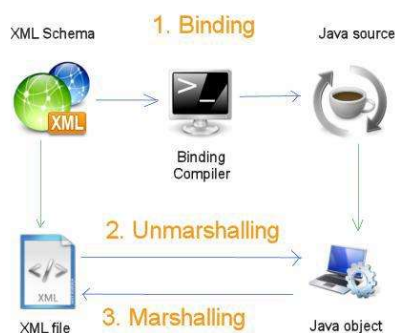
La mise en oeuvre de JAXB 2.0

JAXB 2.0 est une spécification qui permet de mapper des objets Java dans un document XML et vice versa au moyen d'opérations de sérialisation/désérialisation nommées marshalling/unmarshalling. Il permet aussi de générer des classes Java à partir d'un schéma XML et inversement.

Dans les anciennes versions de Java, notamment JDK 8, JAXB était intégré directement au JDK et utilisait le package : `javax.xml.bind`. À partir de JDK 11, JAXB n'est plus intégré au JDK. Il doit donc être ajouté au projet sous forme de dépendances Maven.

Dans cet atelier, nous utiliserons : JDK 23, Maven, Jakarta XML Binding (JAXB 4.x)

Le package utilisé est : `jakarta.xml.bind`



Binding (via le *binding compiler*)

- C'est le processus qui consiste à créer une correspondance (**mapping**) entre un **schéma XML (XSD)** et des **classes Java**.
- C'est grâce au binding qu'on peut transformer facilement un XML en objet Java et inversement.
- Lecture d'un fichier XML pour créer un objet Java.

Le **binding compiler** est l'outil qui effectue cette opération de génération/mapping

Génération XSD → Java

À partir d'un schéma XML un outil de génération appelé **XJC** permet de générer automatiquement les classes Java correspondantes.

Génération Java → XSD

À partir de classes Java annotées JAXB permet de générer automatiquement le schéma XML correspondant.

Remarque : avec JDK 23, les commandes `xjc` et `schemagen` ne sont plus fournies directement par le JDK. Nous utiliserons donc **Maven** et les **outils JAXB externes**.

→ On peut dire que le **binding compiler** = **traducteur** entre le monde XML et le monde Java.

- **Marshalling**
 - C'est la **sérialisation Objet Java → XML**.
 - Conversion d'un objet Java en fichier XML.
- **Unmarshalling**
 - C'est le processus inverse du marshalling
 - C'est la **désérialisation XML → Objet Java**
 - Il permet de **convertir un document XML en objet Java**.

Travail demandé

1. Création du projet

- a. Commencez par créer un projet de type **Maven** et utilisez **JDK 23**.
- b. Vérifiez le fichier **pom.xml** et ajoutez les dépendances suivantes

```
<dependencies>
  <!-- API Jakarta XML Binding -->
  <dependency>
    <groupId>jakarta.xml.bind</groupId>
    <artifactId>jakarta.xml.bind-api</artifactId>
    <version>4.0.2</version>
  </dependency>
  <!-- Implémentation JAXB -->
  <dependency>
    <groupId>org.glassfish.jaxb</groupId>
    <artifactId>jaxb-runtime</artifactId>
    <version>4.0.5</version>
  </dependency>
</dependencies>
```

2. Création de la classe Message.java

- a. Créez la classe **Message.java** comme suit :

```
@XmlRootElement
Public class Message {
private String from;
private String to;
private String text;
private boolean isNew;
}
```

- b. Générer les getters et setters ainsi que les constructeurs (avec et sans paramètres).
- c. **Personnalisation de l'entité** : Ajoutez les annotations JAXB suivantes à la classe « Message ».

```

@XmlRootElement
@XmlType(propOrder={"to","text","from"})
@XmlAccessorType(XmlAccessType.PROPERTY)
Public class Message {
private String from;
private String to;
private String text;
private Boolean isNew;
@XmlElement(name="emetteur",required=true)
public String getFrom() {
return from;
}
Public void setFrom(String from) {
this.from = from;
}
@XmlElement(name="destinataire")
public String getTo() {
return to;
}
Public void setTo(String to) {
this.to = to;
}
@XmlAttribute
public String getText() {
return text;
}
Public void setText(String text) {
this.text = text;
}
@XmlTransient
Public Boolean isNew() {
Return isNew;
}
Public void setNew(Boolean isNew) {
this.isNew = isNew; } }

```

Les annotations utilisées sont :

- **@XmlElement** : Associer la classe à l'élément racine du document XML.
- **@XmlType(propOrder)** : Définir l'ordre d'apparition des éléments dans le document XML
- **@XmlElement** : Convertir un attribut en un élément dans le document XML
- **@XmlAttribute** : Convertir un attribut en un attribut dans le document XML
- **@XmlTransient** : Empêcher la sérialisation d'un attribut
- 3. **@XmlAccessorType** : Spécifie si les annotations JAXB seront situées sur les champs (field) ou sur les getters (property).
- 4. **Génération du fichier xml à partir d'une classe java (Marshalling)**
 - a. Créez la classe suivante qui permet de générer un fichier xml à partir de la classe Message.

- b. Utiliser la classe Marshaller de l'API JAXB pour générer le document XML à partir des objets déjà créés.

```
public class GenererMessageXml {  
    public static void main(String[] args) throws JAXBException {  
        Message message = new Message();  
        message.setFrom("mohamed");  
        message.setNew(true);  
        message.setText("hello");  
        message.setTo("ali");  
        // création d'un contexte JAXB sur la classe Message  
        JAXBContext context = JAXBContext.newInstance(Message.class);  
        // création d'un marshaller à partir de ce contexte  
        Marshaller marshaller = context.createMarshaller();  
        // on choisit UTF-8 pour encoder ce fichier  
        marshaller.setProperty("jaxb.encoding", "UTF-8");  
        // et l'on demande à JAXB de formater ce fichier de façon à pouvoir le lire à l'oeil nu  
        marshaller.setProperty("jaxb.formatted.output", true);  
        // écriture finale du document XML dans un fichier message.xml  
        marshaller.marshal(message, new File("message.xml"));  
    }  
}
```

Le fichier **XML** généré sera situé sous la **racine du projet**.

5. Génération d'une instance java à partir d'un document xml (Unmarshalling)

Créez et exécutez la méthode main suivante :

```
Public static void main(String[] args) throws JAXBException {  
    // création d'un contexte JAXB sur la classe Message  
    JAXBContext context = JAXBContext.newInstance(Message.class) ;  
    // création d'un unmarshaller  
    Unmarshaller unmarshaller = context.createUnmarshaller() ;  
    Message message= (Message)unmarshaller.unmarshal(new File("message.xml"));  
    System.out.println("From = " + message.getFrom());  
    System.out.println("To = " + message.getTo() );  
    System.out.println("Text = " + message.getText() );  
}
```

6. Génération d'un fichier XSD à partir de la classe Message

Créez la classe : GenererSchema.java

```
public class GenererSchema {

    public static void main(String[] args) throws Exception {

        JAXBContext context = JAXBContext.newInstance(Message.class);

        context.generateSchema(new SchemaOutputResolver() {

            @Override
            public Result createOutput(String namespaceUri,
                                     String suggestedFileName)
                throws IOException {

                File file = new File("message.xsd");

                StreamResult result = new StreamResult(file);
                result.setSystemId(file.toURI().toURL().toString());

                return result;    }    });

        System.out.println("Schema généré avec succès : message.xsd");    }}
```

7. Génération d'une classe Message à partir du fichier XSD

Ajoutez le **plugin XJC** dans le fichier pom.xml puis exécutez : mvn clean generate-sources

```
<build>
  <plugins>
    <plugin>
      <groupId>org.codehaus.mojo</groupId>
      <artifactId>jaxb2-maven-plugin</artifactId>
      <version>3.2.0</version>
      <executions>
        <execution>
          <id>xjc</id>
          <goals>
            <goal>xjc</goal>
          </goals>
        </execution>
      </executions>
      <configuration>
        <sources>
          <source>message.xsd</source>
        </sources>
      </configuration>
    </plugin>
  </plugins>
</build>
```